

KYC Face Matching

Project Synopsis

Problem Statement

When companies verify user identities for KYC (Know Your Customer), they usually check a live photo against an ID like a passport. Doing this manually takes too long and costs too much. A lot of modern solutions use massive deep learning models that require heavy servers and tons of memory. I wanted to build something different. The goal was to create a fast, lightweight tool that runs on basic hardware without needing huge external libraries to get the job done.

Tools Used

- C++ for speed and memory control
- stb_image.h for loading images easily without bulky frameworks
- GCC compiler
- Git and GitHub for version control

Method and Process of Development

I broke the development down into a few main steps.

1. Image Parsing

First was getting the images into the program. Instead of bringing in a massive library just to read a photo, I used stb_image.h. It lets the program read standard image files directly from the computer and turns them into raw pixel data the code can actually use.

2. Feature Extraction

Next, I needed a way to find the actual facial features. Just looking at raw colors does not work well. I used a method called HOG (Histogram of Oriented Gradients). The program slides a small window across the image, looking at how the brightness changes from pixel

to pixel. This helps it find edges and shapes, building a structural map of the face that works even if the lighting is a bit off.

3. Vector Comparison

Once the program maps out the faces, it turns those features into math vectors. To compare the two faces, I used two formulas. Mean Squared Error checks the absolute differences between the shapes, and Cosine Similarity checks the angle between the vectors to see how well the overall patterns match.

4. Scoring and Verification

Finally, the program combines these two math scores. If the similarity is high enough and the error is low enough, it flags the two faces as a successful match.